MANUAL DE DESPLIEGUE

DistribuTech Pro — Operaciones, Infraestructura Cloud y Pipelines de CI/CD

Guía Oficial de Puesta en Producción

Proyecto:	Aplicación web B2B
Autor:	Roberto Borrallo Álvarez
Entornos Destino:	Supabase (Database/BaaS) & Vercel (Frontend Hosting)

1. Introducción y Prerrequisitos del Entorno

Este documento detalla de forma exhaustiva la secuencia cronológica de operaciones requerida para realizar la puesta en producción e instanciación limpia de la plataforma **DistribuTech Pro**. El despliegue de la solución se encuentra estructurado bajo un modelo de arquitectura desacoplada moderna: la base de datos relacional y los servicios perimetrales de seguridad se alojan en la nube distribuida de **Supabase**, mientras que el cliente web estructurado en React + TypeScript + Vite se compila y distribuye de forma global a través de la red Edge de alto rendimiento de **Vercel**.

1.1 Prerrequisitos Técnicos

Antes de iniciar el proceso de despliegue, el operador encargado de validar la instalación debe certificar la disponibilidad y correcto funcionamiento local de las siguientes herramientas:

- **Node.js:** Versión activa Long Term Support (LTS) v18.x.x o v20.x.x instalada globalmente. El binario debe incluir el gestor de dependencias de node de forma integrada (`npm`).
- **Git:** Cliente de control de versiones Git para la clonación y sincronización de las ramas del software.
- **Cuentas Cloud Activas:** Un perfil operativo en la plataforma `Supabase.com` y un perfil verificado en `Vercel.com` (asociado preferentemente a la cuenta de GitHub contenedora del código fuente).

2. Proceso de Despliegue Secuencial Paso a Paso

Para garantizar el éxito de la instalación sin incurrir en errores de variables de entorno ni rupturas de claves foráneas, siga meticulosamente los siguientes pasos estructurados:

1 Clonación del Repositorio y Descarga de Dependencias

Abra un terminal local del sistema operativo y proceda a descargar el código fuente oficial desde el repositorio público de GitHub. Acceda al directorio raíz e instale el árbol completo de paquetes especificados en el manifiesto `package.json` :

```
$ git clone https://github.com/robertorayo/DistribuTech-Pro.git
$cd DistribuTech-Pro$ npm install
```

*Nota: La instalación generará la carpeta `node_modules/` . Asegúrese de que no se produzcan advertencias de conflictos de dependencias críticas de TypeScript.

2 Aprovisionamiento y Configuración de Supabase

Inicialice la infraestructura de persistencia siguiendo estas directrices en la consola web de Supabase:

1. Haga clic en **"New Project"**, asigne un nombre al proyecto (ej: *DistribuTech-Production*) y establezca una contraseña segura para el administrador de PostgreSQL. Seleccione la región geográfica más cercana (ej: *West Europe*).
2. Una vez instanciado el aprovisionamiento, navegue al módulo lateral **"SQL Editor"** y cree una nueva consulta vacía.
3. Abra el archivo local `database/schema.sql`, copie la totalidad de su contenido DDL e inyéctelo dentro del editor web. Pulse el botón **"Run"** para generar las tablas, relaciones y triggers de sincronización de perfiles.
4. Abra una nueva pestaña de consulta, copie el contenido del archivo local `database/rls-policies.sql` y ejecútelo para activar de manera inmediata la capa perimetral RLS de seguridad por fila.

3 Configuración de Buckets de Almacenamiento Estático

La carga multimedia de imágenes de productos y fichas técnicas complementarias requiere un espacio aislado en la infraestructura.

- Acceda a la sección **"Storage"** en el panel de control de Supabase.
- Haga clic en **"Create a new bucket"**.
- Establezca obligatoriamente como identificador del bucket el nombre `product-images`.
- Configure el nivel de acceso como **Public** (Público), de modo que los clientes autenticados puedan renderizar los recursos directamente mediante URLs firmadas de lectura, pero restringiendo la subida únicamente a los tokens de administradores según las directrices definidas en las políticas de almacenamiento.

4 Instanciación de Variables de Entorno Locales

Para validar el comportamiento correcto en modo de desarrollo antes del envío definitivo a producción, replique la plantilla de configuración de variables de entorno del sistema:

```
$ cp .env.example .env
```

Edite el archivo `.env` e introduzca los endpoints extraídos de la sección **Project Settings -> API** de su consola de Supabase:

```
VITE_SUPABASE_URL=https://uifozbxytrwqaexample.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InVpZm96Ynh5dHJ3cWEiLCJyb2xlIjoiYW5vbiIsImlhdCI6MTcxOTU5OT...
```

> **ADVERTENCIA DE SEGURIDAD INTERNA:** Jamás añada la clave maestra `SERVICE_ROLE_KEY` dentro del archivo del cliente frontend. Dicha clave anula las comprobaciones RLS y expondría de manera pública el control total de la base de datos en el navegador del cliente.

5 Despliegue de la Aplicación en Vercel (CD Pipeline)

El despliegue definitivo en la nube automatiza el flujo de integración continua ligando la cuenta con el repositorio de software:

1. Acceda a la consola de control de **Vercel** e inicie sesión con su perfil de GitHub.
2. Haga clic en el botón **"Add New..."** y seleccione la opción **"Project"**.
3. En la lista de repositorios importables de su perfil, localice `DistribuTech-Pro` y haga clic en **"Import"**.
4. En la ventana de ajustes de compilación (**Build and Output Settings**), valide que los comandos detectados de forma automática correspondan a las especificaciones estandarizadas de Vite:
   - *Build Command:* `npm run build` o `vite build`
   - *Output Directory:* `dist`
   - *Install Command:* `npm install`
5. Despliegue el acordeón denominado **"Environment Variables"** e inyecte con idéntica nomenclatura las dos variables configuradas previamente en el paso 4 (`VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY`).
6. Haga clic definitivo en el botón **"Deploy"**. El pipeline de Vercel compilará de forma estática el código y asignará una dirección SSL válida de producción (ej: `https://distributech-pro.vercel.app`).

3. Pruebas de Humo y Verificación Post-Despliegue

Una vez finalizado el pipeline de compilación de Vercel, el operador debe certificar la correcta comunicación entre capas ejecutando las siguientes pruebas de humo (*Smoke Tests*) funcionales directas sobre la URL pública de producción:

1. **Verificación de Enrutamiento y Autenticación:** Acceda a la plataforma. El sistema debe redirigirle de forma automática a la vista de `/auth`. Proceda a crear un nuevo usuario mediante el formulario de registro y verifique que el registro se inserte de forma limpia tanto en el módulo interno de Supabase Auth como en la tabla relacional `profiles` a través del trigger automático.
2. **Prueba de Aislamiento de Capa RLS:** Conecte dos navegadores independientes en paralelo (uno con sesión de cliente y otro con sesión de comercial). Inicie una propuesta de negociación de cotización y verifique que los registros no sean legibles ni alterables por ningún perfil ajeno a la relación comercial preestablecida en la matriz relacional.